

GoCD Pipeline — AllianceMine Build & Release

Overview

The AllianceMine build runs on **AllianceMineDev** (172.31.60.197) as a GoCD agent. The pipeline is split into stages that can be manually triggered or chained. The build requires 64 GB RAM and 8+ hours, so it runs on a dedicated agent — not a shared pool.

Agent Setup

GoCD Agent on AllianceMineDev

The GoCD agent must run on AllianceMineDev because: - The build container needs 56 GB RAM (can't run on standard CI agents) - It needs network access to RDS and the Multitenant Solr - It needs AWS credentials for S3 sync and CloudWatch

```
# Install GoCD agent on AllianceMineDev
# Point it at the existing GoCD server
# Assign resource tag: alliancemine-builder
```

Agent Resources/Environments

Resource Tag	Machine	Purpose
alliancemine-builder	172.31.60.197	AllianceMine builds

Pipeline: AllianceMine-Build

Pipeline Settings

Setting	Value
Group	intermine
Material	git: alliance-genome/agr_intermine_builder, branch dev
Label template	\${COUNT}-\${ALLIANCE_RELEASE}
Environment	ALLIANCE_RELEASE, BUILD_TYPE, RC_NUMBER
Lock behavior	Locked (only one build at a time)
Timeout	12 hours

Environment Variables

Variable	Default	Secure	Description
ALLIANCE_RELEASE	(required)	No	Release version (e.g., 9.0.0)
BUILD_TYPE	test	No	test or production
RDS_PASSWORD		Yes	RDS PostgreSQL password
SGD_DB_PASSWORD		Yes	SGD database password
SUPERUSER_PASSWORD		Yes	InterMine superuser password
SOLR_HOST	172.31.59.87	No	Multitenant Solr host
TOMCAT_HOST	172.31.59.87	No	Multitenant Tomcat host
TOMCAT_PORT	8082	No	Deployment Tomcat port

Stage 1: check-prerequisites

Type: Manual trigger **Agent:** alliancemine-builder **Tasks:**

Task 1: Check FMS release data availability

```
python3 -c "  
import urllib.request, json  
resp = url-  
↳ lib.request.urlopen('https://fms.alliancegenome.org/api/snapshot/release/${ALLIANCE_RELEASE}')
```

Task 2: Check RDS free storage

```
FREE_GB=$(aws cloudwatch get-metric-statistics --namespace AWS/RDS \  
--metric-name FreeStorageSpace \  
--dimensions Name=DBInstanceIdentifier,Value=intermine-postgres \  
--start-time $(date -u -d '5 minutes ago' +%Y-%m-%dT%H:%M:%S) \  
--end-time $(date -u +%Y-%m-%dT%H:%M:%S) \  
--period 300 --statistics Average \  
--query 'Datapoints[0].Average' --output text | awk '{printf "%.0f",  
↳ $1/1024/1024/1024}')
```

Task 3: Check SGD database connectivity

```
PGPASSWORD="${SGD_DB_PASSWORD}" psql -h www-rds-primary.yeastgenome.org \  
-p 5432 -U webb -d sgd -c "SELECT 1" || { echo "FAIL: SGD DB unreachable"; exit 1; }
```

Stage 2: create-solr-cores

Type: Auto (runs after prerequisites pass) **Agent:** alliancemine-builder **Tasks:**

```
cd docker/alliancemine  
python3 scripts/create_solr_cores.py \  
--release ${ALLIANCE_RELEASE} \  
--solr-host ${SOLR_HOST} \  
--ssh-key ~/.ssh/AGR-ssl3.pem
```

Stage 3: build-image

Type: Auto **Agent:** alliancemine-builder **Tasks:**

```
cd docker/alliancemine  
  
# Write .env from GoCD env vars  
cat > .env << ENVEOF  
RDS_HOST=intermine-postgres.cmnhlso7wdi.us-east-1.rds.amazonaws.com  
RDS_PORT=5432  
RDS_USER=postgres  
RDS_PASSWORD=${RDS_PASSWORD}  
ALLIANCE_RELEASE=${ALLIANCE_RELEASE}  
BUILD_TYPE=${BUILD_TYPE}  
SGD_DB_HOST=www-rds-primary.yeastgenome.org  
SGD_DB_NAME=sgd
```

```

SGD_DB_USER=webb
SGD_DB_PASSWORD=${SGD_DB_PASSWORD}
SGD_DB_PORT=5432
SOLR_INDEX_URL=http://${SOLR_HOST}:8983/solr/alliancemine-search-${ALLIANCE_RELEASE}
SOLR_AUTOCOMPLETE_URL=http://${SOLR_HOST}:8983/solr/alliancemine-autocomplete-
  ↪ ${ALLIANCE_RELEASE}
SUPERUSER_ACCOUNT=superuser@mail_account
SUPERUSER_PASSWORD=${SUPERUSER_PASSWORD}
WEBAPP_BASEURL=http://localhost:8080
IMAGE_TAG=${ALLIANCE_RELEASE}
ENVEOF

```

```

# Build image
docker compose build

```

```

# Compile and save
docker compose run -d --rm alliancemine-builder
CNAME=$(docker ps --format '{{.Names}}' | grep builder)
while docker exec $CNAME ls /root/.needs_compile 2>/dev/null; do sleep 30; done
docker commit $CNAME alliancemine-builder:${ALLIANCE_RELEASE}-compiled
docker stop $CNAME

```

```

# Update compose to use compiled image
sed -i "s|image: alliancemine-builder:.*|image:
  ↪ alliancemine-builder:${ALLIANCE_RELEASE}-compiled|" docker-compose.yml

```

Stage 4: run-build

Type: Auto **Agent:** alliancemine-builder **Timeout:** 10 hours **Tasks:**

```

cd docker/alliancemine
docker compose run --rm alliancemine-builder build \
  2>&1 | tee /tmp/alliancemine-${ALLIANCE_RELEASE}-build.log

```

```

# Verify build succeeded
tail -5 /tmp/alliancemine-${ALLIANCE_RELEASE}-build.log | grep -q "BUILD COMPLETED" || {
  echo "BUILD FAILED"
  tail -30 /tmp/alliancemine-${ALLIANCE_RELEASE}-build.log
  exit 1
}

```

Artifacts: /tmp/alliancemine-\${ALLIANCE_RELEASE}-build.log □ publish as build artifact

Stage 5: fix-solr-indexes

Type: Auto **Agent:** alliancemine-builder **Tasks:**

```

cd docker/alliancemine

```

```

# Start container from compiled image
docker compose run -d --rm -e RC_NUMBER=${RC_NUMBER} alliancemine-builder
CNAME=$(docker ps --format '{{.Names}}' | grep builder)

```

```

# Wait for compilation/entrypoint
sleep 30

```

```

# Patch hardcoded Solr URLs (until upstream PR is merged)

```

```

docker exec $CNAME sed -i \
"s|index.solrurl = .*|index.solrurl =
↪ http://${SOLR_HOST}:8983/solr/alliancemine-search-${ALLIANCE_RELEASE}|" \
/root/alliancemine/dbmodel/resources/keyword_search.properties

docker exec $CNAME sed -i \
"s|autocomplete.solrurl = .*|autocomplete.solrurl =
↪ http://${SOLR_HOST}:8983/solr/alliancemine-autocomplete-${ALLIANCE_RELEASE}|" \
/root/alliancemine/dbmodel/resources/objectstoresummary.config.properties

# Run search index
docker exec $CNAME sh -c 'cd /root/alliancemine && ./gradlew postprocess
↪ -Pprocess=create-search-index --stacktrace'

# Run autocomplete index
docker exec $CNAME sh -c 'cd /root/alliancemine && ./gradlew postprocess
↪ -Pprocess=create-autocomplete-index --stacktrace'

# Verify
ssh -i ~/.ssh/AGR-ssl3.pem ec2-user@${SOLR_HOST} "python3 -c \"
import urllib.request, json
for core in ['alliancemine-search-${ALLIANCE_RELEASE}',
↪ 'alliancemine-autocomplete-${ALLIANCE_RELEASE}']:
    resp =
↪ urllib.request.urlopen(f'http://localhost:8983/solr/{core}/select?q=*&rows=0')
    data = json.loads(resp.read())
    docs = data['response']['numFound']
    print(f'{core}: {docs} docs')
    assert docs > 0, f'{core} is empty'
\""

docker stop $CNAME

```

Stage 6: deploy

Type: Manual trigger (requires approval) **Agent:** alliancemine-builder **Tasks:**

cd docker/alliancemine

```

# Start Tomcat container on multitenant
ssh -i ~/.ssh/AGR-ssl3.pem ec2-user@${TOMCAT_HOST} \
"docker run -d --name alliancemine-${ALLIANCE_RELEASE} -p ${TOMCAT_PORT}:8080
↪ intermine-tomcat:latest"

# Deploy WAR
docker compose run --rm alliancemine-builder release \
--tomcat-host ${TOMCAT_HOST} \
--tomcat-port ${TOMCAT_PORT}

# Verify
sleep 30
ssh -i ~/.ssh/AGR-ssl3.pem ec2-user@${TOMCAT_HOST} \
"docker exec alliancemine-${ALLIANCE_RELEASE} sh -c 'curl -s
↪ http://localhost:8080/alliancemine/service/version'"

```

Stage 7: cutover

Type: Manual trigger (requires approval) **Agent:** alliancemine-builder **Tasks:**

```
# Update Caddy/proxy to point to new container
# This is environment-specific - update as needed
ssh -i ~/.ssh/AGR-ssl3.pem ec2-user@${TOMCAT_HOST} \
    "echo 'Cutover: update proxy to port ${TOMCAT_PORT}'"
```

Pipeline Visualization

check-prerequisites → create-solr-cores → build-image → run-build → fix-solr → [APPROVE] → deploy → [APPROVE]

(manual) (auto) (auto) (auto,10h) (auto) (manual) (manual)

Cleanup Pipeline: AllianceMine-Cleanup

Separate pipeline, triggered manually after successful release.

Stage 1: cleanup-checkpoints

```
# Drop checkpoint databases for the completed RC
PGPASSWORD="${RDS_PASSWORD}" psql \
    -h intermine-postgres.cmnhlso7wdi.us-east-1.rds.amazonaws.com \
    -U postgres -d postgres -c \
    "SELECT datname FROM pg_database WHERE datname LIKE 'alliancemine_%rc%:%'" \
    -tA | while read db; do
    echo "Dropping: $db"
    PGPASSWORD="${RDS_PASSWORD}" psql \
        -h intermine-postgres.cmnhlso7wdi.us-east-1.rds.amazonaws.com \
        -U postgres -d postgres -c "DROP DATABASE \"$db\""
done
```

Stage 2: cleanup-old-rcs

```
# Drop old RC databases (keep production version)
PGPASSWORD="${RDS_PASSWORD}" psql \
    -h intermine-postgres.cmnhlso7wdi.us-east-1.rds.amazonaws.com \
    -U postgres -d postgres -c \
    "SELECT datname FROM pg_database WHERE datname LIKE 'alliancemine_%rc%' AND datname
    ↪ NOT LIKE '%:%'" \
    -tA | while read db; do
    echo "Dropping: $db"
    PGPASSWORD="${RDS_PASSWORD}" psql \
        -h intermine-postgres.cmnhlso7wdi.us-east-1.rds.amazonaws.com \
        -U postgres -d postgres -c "DROP DATABASE \"$db\""
done
```

Stage 3: cleanup-docker

```
# Remove old compiled images
docker images | grep 'alliancemine-builder' | grep -v ${ALLIANCE_RELEASE} | awk '{print
    ↪ $3}' | xargs -r docker rmi

# Remove old containers on multitenant
ssh -i ~/.ssh/AGR-ssl3.pem ec2-user@${TOMCAT_HOST} \
```

```
"docker ps -a --filter name=alliancemine- --format '{{.Names}}' | grep -v  
↪  ${ALLIANCE_RELEASE} | xargs -r docker rm -f"
```

Scheduled Trigger (Optional)

To auto-check for new Alliance releases:

```
# GoCD timer trigger - weekly check  
timer:  
  spec: "0 0 8 ? * MON" # Every Monday at 8 AM  
  only_on_changes: false
```

Add a task to Stage 1 that checks FMS for a new release version and only proceeds if it's newer than the last build.

Security Notes

- All credentials (RDS_PASSWORD, SGD_DB_PASSWORD, SUPERUSER_PASSWORD) must be GoCD **secure environment variables**
- SSH key ~/.ssh/AGR-ssl3.pem must be on the GoCD agent machine
- Never log credential values in build output
- The .env file is generated per-build and should be excluded from artifacts